Usage Guide

How to run G1Control in simulation and on the real Unitree G1, how to check gesture recognition, and how to re-record gestures. The install must already be complete; see SETUP.md.

For an explanation of how the program and its components fit together, see the README.

Contents

1. Before each session
2. Verify gesture recognition
3. Run in simulation
4. Run on the real G1
5. Re-record gestures
6. Troubleshooting

1. Before each session

Activate the Python environment in every new terminal before running any project command:

```
source ~/HumanoidControl/Dependencies/g1-env/bin/activate
```

Camera colours. Some USB cameras (including the Insta360) show corrupted colours in the program the first time they are used after being plugged in. If this happens, open the Cheese webcam app once, confirm a normal picture appears, close it, then start the program. Install Cheese with `sudo apt install cheese` if needed.

2. Verify gesture recognition

Test the camera and gesture recognition on their own, without the simulator or robot:

```
cd ~/HumanoidControl/G1Control
python src/recognize_hand_gestures.py
```

A camera window opens with a skeleton drawn over your hand. Make the gestures shown in `gesture_library/`; the recognised gesture name is printed in the terminal each time it changes. Press `q` to quit.

To make matching looser or stricter, pass a threshold (higher is looser):

```
python src/recognize_hand_gestures.py --threshold 2.0
```

3. Run in simulation

3.1 Start the three programs

The simulation needs three programs running at the same time, each in its own terminal. **Activate the virtual environment in each terminal** before running its command.

Start them **in this order**: `run.py` creates the virtual gamepad, and the simulator only looks for a gamepad when it starts.

Terminal 1: the program

```
source ~/HumanoidControl/Dependencies/g1-env/bin/activate
cd ~/HumanoidControl/G1Control
python run.py sim
```

Wait until it prints `Virtual gamepad is live`.

Terminal 2: the simulator

```
source ~/HumanoidControl/Dependencies/g1-env/bin/activate
cd ~/HumanoidControl/Dependencies/unitree_mujoco/simulate/build
./unitree_mujoco
```

Terminal 3: the walking controller

```
source ~/HumanoidControl/Dependencies/g1-env/bin/activate
cd ~/HumanoidControl/Dependencies/unitree_rl_lab/deploy/robots/g1_29dof/build
./g1_ctrl --network lo
```

Wait until it prints `Connected to robot.`

3.2 Start-up sequence

Return to Terminal 1:

1. Press **Enter**. The robot moves into its standing pose.
2. The program lowers the support band in steps and asks `Are the robot's feet on the ground?`. Watch the MuJoCo window: press **Enter** to lower further, or type **y** and press Enter once both feet are flat on the floor. Keep the MuJoCo window visible and uncovered during this step; the program sends key presses to it.
3. The program starts the walking policy and releases the band.
4. A camera window titled **Humanoid Control** opens.

3.3 Controls

In the camera window:

Key	Action
`m`	Switch between gesture mode and mimic mode

Key	Action
`q`	Quit

3.4 Gesture mode

The program starts in gesture mode. Hold a gesture to move; lower your hand to stop. Photos of each gesture are in `gesture_library/`.

Gesture	Robot action
`move_forward_left_hand`, `move_forward_right_hand`	Walk forward
`move_backward_left_hand`, `move_backward_right_hand`	Walk backward
`move_left`	Step sideways to the left
`move_right`	Step sideways to the right
`turn_left`	Turn left
`turn_right`	Turn right

The status line at the top of the camera window shows the recognised gesture, or `no match`.

To make matching looser or stricter, add a threshold after `sim` (default 1.5, higher is looser):

```
python run.py sim 2.0
```

3.5 Mimic mode

Press `m` to switch. Stand back so your upper body is fully in frame; the robot stops walking and its arms follow yours. Press `m` again to return to gesture mode.

- The first switch takes a few seconds while the models load, and the camera image freezes during that time.
- Move your arms at a moderate pace. Arm speed is capped because fast two-arm movements can tip the robot over.

3.6 Shutting down

Press `q` in the camera window, then press `Ctrl+C` in Terminals 3 and 2.

---

4. Run on the real G1

> **Untested.** This mode has not yet been tested on a physical robot. The steps below describe how the code is designed to work; expect to troubleshoot on the first attempt.

> **Safety.** Have someone holding the Unitree remote with the emergency stop ready, keep at least 3 m clear around the robot, and use a gantry or harness for the first runs.

On the real robot, the program sends walking commands directly to the G1's built-in walking controller. The simulator and `g1_ctrl` are not used. **Only gesture mode is available on the real robot.**

1. **Connect to the same WiFi network as the robot.** The computer and the G1 must be on the same network for the program's commands to reach the robot. Note the robot's IP address on that network (for example from your router or the Unitree app), then check that the computer can reach it:

```
ping <robot-ip-address>
```

Replies such as `64 bytes from <robot-ip-address>` mean the connection works. Press `Ctrl+C` to stop.

2. **Find the WiFi interface name.** Run:

```
ip -br addr
```

Note the name of the interface that starts with `wl` and has an IP address, for example `wlp2s0`. On most Ubuntu laptops it is **not** `wlan0`.

3. **Set the interface in the code.** Open `src/real_controller.py` and change `wlan0` in this line to your interface name:

```
def __init__(self, interface="wlan0"):
```

4. **Prepare the robot.** Power on the G1 and use the handheld remote to put it into its normal standing walking mode, as described in Unitree's G1 user manual. The robot must already be standing and able to walk from the remote.

5. **Run:**

```
source ~/HumanoidControl/Dependencies/g1-env/bin/activate
cd ~/HumanoidControl/G1Control
python run.py real
```

Walking and turning speeds are deliberately lower than in simulation (0.3 m/s and 0.3 rad/s). Lowering your hand, or losing the gesture, sends a stop command. Pressing `q` stops the robot before exiting.

5. Re-record gestures

The gesture data in `src/data/gestures.json` was recorded by the development team. If recognition is unreliable for a new operator, camera or lighting, record new samples:

```
cd ~/HumanoidControl/G1Control
python src/record_hand_gestures.py
```

Hold a gesture and press its number key; 20 samples are captured. Press `s` to save and quit.

Key	Gesture
`1`	`turn_right`
`2`	`turn_left`

Key	Gesture
`3`	`move_right`
`4`	`move_left`
`5`	`move_forward_left_hand`
`6`	`move_forward_right_hand`
`7`	`move_backward_left_hand`
`8`	`move_backward_right_hand`

New samples are **added** to the existing file. To start from scratch, back up and delete `src/data/gestures.json` first, then record all 8 gestures before pressing `s`.

Check the result with Verify gesture recognition.

---

6. Troubleshooting

Problem	Fix
`python: command not found`	The environment is not active in this terminal. Run the `source` command from Before each session.
`g1_ctrl` never prints `Connected to robot.`	The simulator is not using the project's settings. Re-copy `config.yaml` as in SETUP.md step 11.1, then restart the simulator. Make sure `g1_ctrl` was started with `--network lo`.
Robot does not respond to the start-up sequence or to gestures	The simulator did not find the gamepad. Restart in the correct order: `run.py` first, then the simulator. If a physical controller is also connected, run `ls /dev/input/js*`; if the virtual gamepad is `js1`, change `joystick_device` to `"/dev/input/js1"` in `Dependencies/unitree_mujoco/simulate/config.yaml`.
Support band does not lower or release	The program cannot send key presses to the MuJoCo window. Check that you are using Xorg (see SETUP.md) and that the MuJoCo window is open and uncovered. As a fallback, click the MuJoCo window and press `8` to lower the band and `9` to release it.
`Could not open camera`, or a black camera window	Check that `ls /dev/video*` lists a device, and close any other app using the camera.
Wrong camera opens	The program uses the first camera. Unplug other cameras or disable the built-in one.
Colours look wrong	Open and close Cheese once (see Before each session).
Mimic mode prints `Couldn't start MimicSource`	The program continues in gesture mode. Read the error after the colon; usually `pose_landmarker.task` is missing from `Dependencies/Models/`.
Arms do not move in mimic mode	The simulator was built without the project's patch. Redo SETUP.md step 11 and rebuild.
Robot falls over	Restart all three programs. In mimic mode, move your arms more slowly.
Simulator runs in slow motion	The CPU is overloaded. Close other programs.

Problem	Fix
Real robot does not move	Check that the computer and robot are on the same WiFi network (`ping` the robot), that the interface name in `src/real_controller.py` matches your WiFi interface, and that the robot is in standing walking mode. Real-robot mode is untested; see the note in Section 4.
Simulator closes immediately with `Error: Joystick open failed.` or `Segmentation fault`	The virtual gamepad does not exist yet. Start `run.py` first and wait for `Virtual gamepad is live`, then start the simulator.
Mimic mode fails with `/tmp/cdds.LOG: cannot open for writing`	Another user account on this computer has run the program before and owns the DDS log file. Reboot (Ubuntu clears `/tmp` on start-up), or delete the file with `sudo rm /tmp/cdds.LOG`, then try again.